Lab 3 — Batch A

Scheme (DrRacket, #lang sicp) · 14:00–15:25

Language: Scheme (DrRacket, `#lang sicp`)

Assume the following are already available to you exactly as defined below — copy this block into your file. `map` is already built into your language, so just use it directly; use *these* definitions of `length` / `filter` / `foldr` / `foldl` rather than reaching for any other version.

```
(define (length lst)
  (define (length-iter n lst) (if (null? lst) n (length-iter (+ 1 n) (cdr lst))))
  (length-iter 0 lst))

(define (filter pred l)
  (if (null? l) nil
      (let ((x (car l)))
        (if (pred x) (cons x (filter pred (cdr l))) (filter pred (cdr l))))))

(define (foldr f v l)
  (if (null? l) v (f (car l) (foldr f v (cdr l)))))

(define (foldl f v l)
  (if (null? l) v (foldl f (f v (car l)) (cdr l))))
```

Q1 (1 mark) — `count-after-curve` (file name: q1.scm)

Write a procedure `count-after-curve` that takes a list of exam scores, a curve amount, and a cutoff, and returns how many scores would be at or above the cutoff **after** adding the curve to every score. Use `map` together with `filter` and `length` — don't write a hand-rolled recursive loop.

```
(count-after-curve '(30 45 60 75) 10 50) => 3
(count-after-curve '(10 20) 5 50)        => 0
```

Q2 (1 mark) — `average-passing-score` (file name: q2.scm)

Write a procedure `average-passing-score` that takes a list of exam scores and a cutoff, and returns the average of just the scores at or above the cutoff. Assume **at least one** score meets the cutoff, so you don't need to handle dividing by zero. Use `filter` together with `foldr` (and `length`) — don't write a hand-rolled recursive loop.

```
(average-passing-score '(60 70 80) 50)      => 70
(average-passing-score '(40 60 80 100) 70)  => 90
```

Q3 (1 mark) — `doubled-above-cutoff-reversed` (file name: q3.scm)

Write a procedure `doubled-above-cutoff-reversed` that takes a list of scores and a cutoff, and: keeps only the scores strictly above the cutoff, doubles each of them, and returns the result **in reverse of the order they'd naturally come out** — using `filter`, `map`, and `foldl` (specifically `foldl`, not `foldr`, and not a built-in `reverse`). Think about how `foldl`'s accumulator gets threaded through, and what happens if you `cons` each element onto it as you go.

```
(doubled-above-cutoff-reversed '(10 25 5 40) 20) => (80 50)
(doubled-above-cutoff-reversed '(10 20) 50)      => ()
```

(Sanity check: `filter` + `map` alone, without the fold, would give you `(50 80)` — in forward order. Your job is to also flip that order, using nothing but `foldl`.)

Take-home practice (ungraded)

`total-shortfall`

Write `total-shortfall`, which takes a list of exam scores and a cutoff, and returns the **total number of additional points** needed across all the *failing* scores to bring each one up to the cutoff (e.g. a score of 35 with a cutoff of 50 needs 15 more points). Use `filter`, `map`, and `foldr`.

```
(total-shortfall '(35 72 40 55) 50) => 25   ; 15 + 10
```

Once you're done, feel free to also try Batch B's Q1–Q3 for extra practice.